CVSD Final Project - BCH Decoder

b11901172　陳繕禎

b11901043　張庭碩

Algorithm Design

1. Syndrome Computation

The computation of the required 2t syndromes is performed as follows:

First, the received codeword r(x) is divided by the corresponding minimal polynomial Φ(x), and the remainder polynomial b(x) is obtained.

The syndromes are then calculated by evaluating this remainder polynomial at successive powers of the primitive element, namely α, α^2.....α^{2t}. Since the remainder polynomial has a degree lower than that of the minimal polynomial, this approach significantly reduces the computational complexity compared to directly evaluating the received codeword.

2. Simplified Berlekamp

For binary BCH codes, the Berlekamp algorithm can be simplified by exploiting the property that squaring a polynomial over GF(2) is equivalent to evaluating it at x^2. This leads to the syndrome relationship $S^2(x) = S(x^2)$, meaning that if the odd-indexed Newton identities are satisfied, the corresponding even-indexed identities are automatically satisfied.

As a result, every pair of iterations in the standard Berlekamp algorithm produces the same error-location polynomial. Therefore, only the odd-indexed steps are required, reducing the total number of iterations from 2t to t. This halves the computational complexity while maintaining correct error-location capability for binary BCH codes.

3. Berlekamp 2^t- Case Expansion

In the conventional Berlekamp–Massey algorithm, the error-locator polynomial σ(x) is updated iteratively for a total of 2t discrepancy evaluations, corresponding to the required 2t syndromes. Each iteration depends on the previous result, which leads to a strictly sequential computation.

In this design, we introduce a 2^t-case expansion optimization by exploiting the algebraic property of binary BCH codes, where even-indexed syndromes are functionally dependent on odd-indexed syndromes. As a result, each pair of iterations (2μ-1, 2μ) produces the same error-locator polynomial.

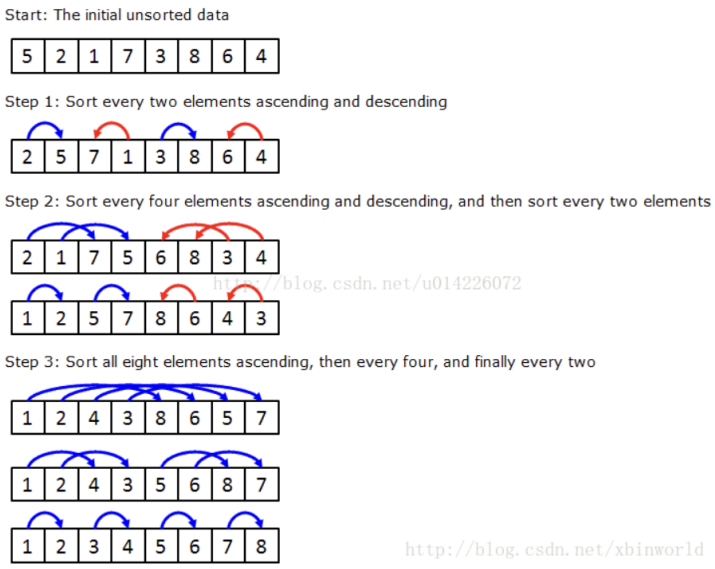
Based on this observation, the original 2t sequential iterations are reformulated into 2^t expanded cases, where each case evaluates two discrepancies in parallel. The update logic is fully unrolled and expressed as fixed GF(2^m) arithmetic, eliminating runtime branching and iterative dependency.

This expansion enables parallel evaluation of candidate updates and significantly reduces decoding latency, making the Berlekamp procedure more suitable for high-throughput hardware implementation.

4. Bitonic Sorter

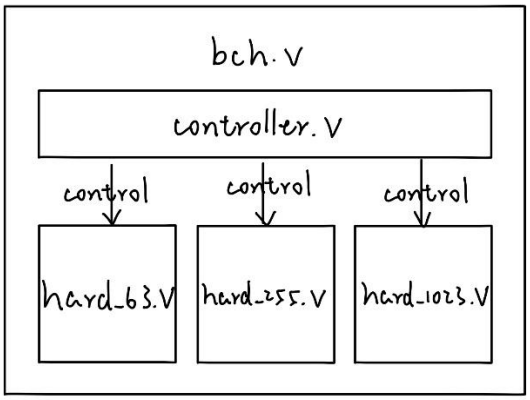
Bitonic sorting is used to find the least reliable bits and sorts error locations in ascending order. Bitonic sorting is very advantageous in terms of area and the complexity of the control logic since it only needs 4 adders and some multiplexers to sort 8 data.

The sorting procedure is provided as follow:



Hardware Implementation

Design hierarchy



(1)Hard-decision decoding submodule implementation

1. GF Division for b(x) : 8-way loop unrolled divider

To compute the remainder polynomials b1(x) , b2(x) ... required for syndrome generation, we implement a parallel LFSR-based polynomial divider over GF(2^m).

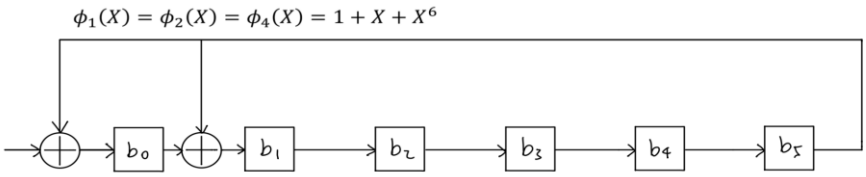
Because the received hard-decision data arrives 8 bits per cycle, the divider is 8-way loop-unrolled, allowing one full byte to be processed in a single cycle.

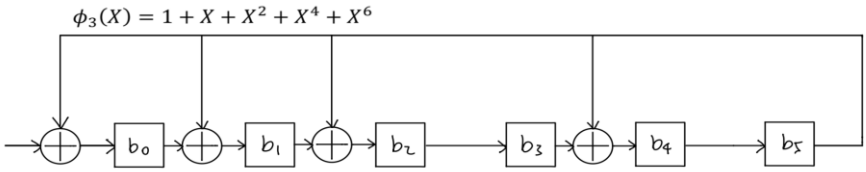
Take 63-51 code for example,

Two divider instances are implemented in parallel:

Divider 1 computes b1(x), b2(x), b4(x)

Divider 2 computes b3(x)





Each divider is realized as a 6-bit shift register with XOR feedback, where the feedback taps are determined by the minimal polynomial.

This architecture significantly reduces latency compared to a serial divider.

2. Syndrome Computing : Replacing GF Exponentiation

To compute the syndromes, the powers of the primitive element α^i are symbolically substituted into the remainder polynomial and fully expanded using the primitive polynomial of the field.

After expansion, each syndrome can be expressed as a fixed XOR combination of the remainder coefficients. As a result, syndrome computation is implemented as a pure combinational XOR network and can be completed within a single clock cycle without loop unrolling or field exponentiation.

$$S_1 = b_1(\alpha^1) = b_0 + b_1\alpha + b_2\alpha^2 + b_3\alpha^3 + b_4\alpha^4 + b_5\alpha^5$$

$$S_2 = b_2(\alpha^2) = b_0 + b_1\alpha^2 + b_2\alpha^4 + b_3\alpha^6 + b_4\alpha^8 + b_5\alpha^{10} = b_0 + b_1\alpha^2 + b_2\alpha^4 + b_3(\alpha+1) + b_4(\alpha^3+\alpha^2) + b_5(\alpha^5+\alpha^4)$$
$$= (b_0+b_3) + b_3\alpha + (b_1+b_4)\alpha^2 + b_4\alpha^3 + (b_2+b_5)\alpha^4 + b_5\alpha^5$$

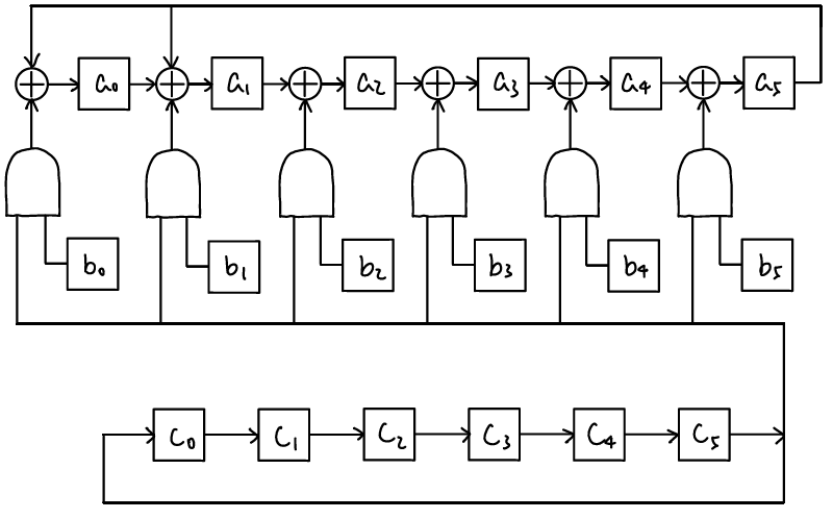
$$S_3 = b_3(\alpha^3) = b_0 + b_1\alpha^3 + b_2\alpha^6 + b_3\alpha^9 + b_4\alpha^{12} + b_5\alpha^{15} = b_0 + b_1\alpha^3 + b_2(\alpha+1) + b_3(\alpha^4+\alpha^3) + b_4(\alpha^2+1) + b_5(\alpha^5+\alpha^3)$$
$$= (b_0+b_2+b_4) + b_2\alpha + b_4\alpha^2 + (b_1+b_3+b_5)\alpha^3 + b_3\alpha^4 + b_5\alpha^5$$

$$S_4 = b_4(\alpha^4) = b_0 + b_1\alpha^4 + b_2\alpha^8 + b_3\alpha^{12} + b_4\alpha^{16} + b_5\alpha^{20}$$
$$= b_0 + b_1\alpha^4 + b_2(\alpha^3+\alpha^2) + b_3(\alpha^2+1) + b_4(\alpha^4+\alpha+1) + b_5(\alpha^5+\alpha^4+\alpha^3+\alpha^2)$$
$$= (b_0+b_3+b_4) + b_4\alpha + (b_2+b_3+b_5)\alpha^2 + (b_2+b_5)\alpha^3 + (b_1+b_4+b_5)\alpha^4 + b_5\alpha^5$$

The figure above takes 63-51 code as an example to show how we expand the higher order GF elements to basic vectors, so that we can easily compute the syndromes.

3. GF Multiplier : Loop unrolled

The original multi-cycle GF(2^m) multiplier is optimized by fully unrolling the LFSR-based computation into a single-cycle combinational XOR network based on the primitive polynomial, thereby eliminating iterative feedback and significantly reducing decoding latency.



Feedback signals are connected according to primitive polynomial.

4. GF(2^m) inverse computation

The inverse operation in GF(2^m) is implemented using Fermat' s theorem. Since $\alpha^n = 1$ · $\alpha^{-1} = \alpha^{n-1}$. Take n=63 for example, $\alpha^{-1} = \alpha^{n-1} = \alpha^{62} = \alpha^2 \cdot \alpha^4 \cdot \alpha^8 \cdot \alpha^{16} \cdot \alpha^{32}$.

This is realized by a sequence of squaring and multiplication operations. Squaring is implemented as a combinational XOR-based circuit exploiting the special property $S^2(x) = S(x^2)$ in GF(2), while the multiplier accumulates the required products.

The inverse is obtained after a fixed number of cycles, making the design area-efficient and well-suited for hardware implementation.

5. Root Finding : 4-way parallel acceleration

The Chien search is implemented using a four-way parallel architecture that evaluates four consecutive powers of the primitive element per cycle.

By pre-expanding the field multiplications and unrolling the GF(2^m) operations into combinational XOR networks, the design eliminates iterative feedback and enables constant-time root evaluation.

This approach reduces the root-finding latency by a factor of four while maintaining low hardware complexity, making it well suited for optimized BCH decoding.

(2)Soft-decision decoding using controller

We design a controller to implement the soft-decision decoding. The procedures are illustrated as states.

$$\text{S_IDLE} \xrightarrow{set} \text{S_LEAST} \xrightarrow{finish\ sending\ r(x)\ to\ submodules} \text{S_WAIT} \xrightarrow{hard-decision\ decoding\ finished} \text{S_LRB}$$

$$\xrightarrow{bitonic\ sorting\ finished} \text{S_FLIP} \xrightarrow{finish\ sending\ r(x)\ to\ submodules} \text{S_WAIT} \xrightarrow{hard-decision\ decoding\ finished}$$

$$\text{S_GAP} \rightarrow \text{S_FLIP} \xrightarrow{finish\ sending\ r(x)\ to\ submodules} \text{S_WAIT} \xrightarrow{hard-decision\ decoding\ finished} \text{S_GAP} \rightarrow$$

$$\text{S_FLIP} \xrightarrow{finish\ sending\ r(x)\ to\ submodules} \text{S_WAIT} \xrightarrow{hard-decision\ decoding\ finished} \text{S_CORR}$$

$$\xrightarrow{finish\ calculating\ correlation} \text{S_DELAY} \rightarrow \text{S_SORT} \xrightarrow{bitonic\ sorting\ finished} \text{S_FINISH} \rightarrow \text{S_IDLE}$$

S_IDLE: wait for set signal

S_LEAST: sending idata to hard-decision decoding submodule and collect least reliable bits concurrently

S_WAIT: Wait for hard-decision decoding finishes.

S_LRB: Sort the collected 8 minimum LLRs using bitonic sorting and find the least reliable

bits.

S_FLIP: Send the flipped r(X) to hard-decision decoding submodule, the flipped bits are least reliable bits.

S_GAP: Redundant state inserted between S_WAIT and S_FLIP due to enable signal setting of hard-decision decoding submodule.

S_CORR: We designed a adder tree that takes 8 LLR as input in one cycle. LLR register shift its data and feeds the adder tree. The computation takes (n+1)/8 cycles.

S_DELAY: Redundant state added between S_CORR and S_SORT to update the chosen pattern.

S_SORT: Sort error location in ascending order.

S_FINISH: Output results.

1. Finding least reliable bits

We designed four groups of comparison units, each takes two LLR and the index of least reliable bits. The comparison is done during S_LEAST while controller is receiving idata[7:0] from bch.v and send the bits to hard-decision decoding submodules. Finding least reliable bits and computing hard-decision decoding concurrently saves (n+1)/8 cycles, but requires lots of adders to find the least reliable bits.

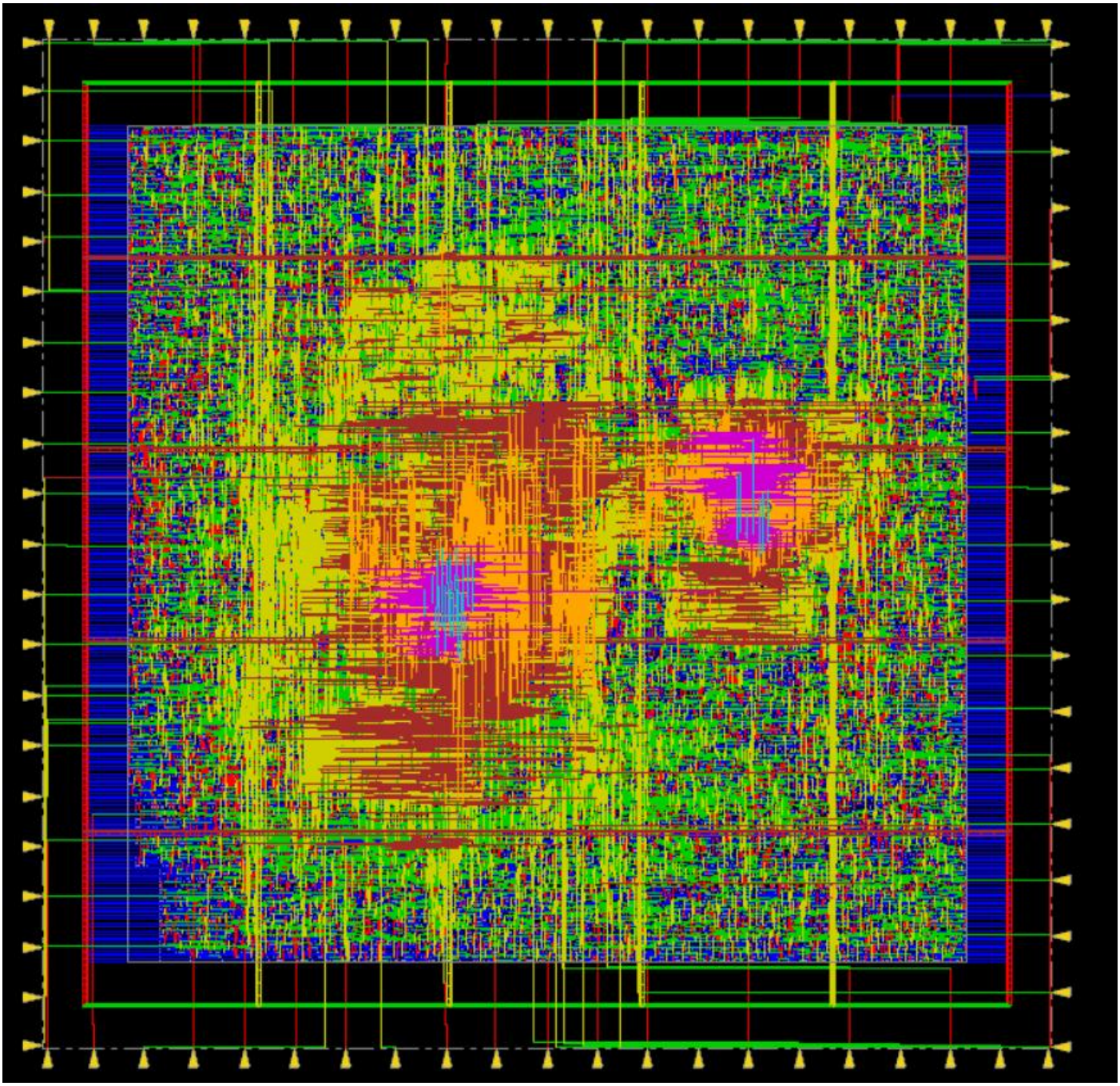
After hard-decision decoding finished, collected minimum LLRs are sent to registers of bitonic sorter. Let the sorter decide which 2 bits are least reliable.

2. Correlation calculation

We created four sets of registers to collect error locations of hard-decision decoding and

flipped least reliable bits. To reduce the design complexity, all unused error location registers are set to 11’d1024. When comparing a LLR index to a unused error location, the result must be false, and the corresponding LLR will not be selected as the adder tree’s input.

APR Results



Performance Evaluation

1. Speed

(1) Clock cycles

Hard-decision decoding

module	hard_63.v	hard_255.v	hard_1023.v
Find b(X)	8	32	128
Syndrome	1	1	1
Berlekamp	7	9	32
Root	16	64	256
Total	32	106	417

Since we utilized 4-way parallel root calculation, the required computation cycles is greatly reduced.

Soft-decision decoding

Code type	n=63	n=255	n=1023
Decoding	128	424	1668
Sorting	14	14	14
Correlation	8	32	128
Total	150	470	1810

This is a rough worst-case evaluation.

(2)Clock period

The clock period is 6.5ns. It is higher than my expectation. The critical path is the adder tree

used to calculate the correlation and its associative input control logic. It envolves roughly 73 bits of addition and some multiplexers. I would design a pipelined adder tree if I had enough time.

2. Power

We utilized clock gating to reduce the registers' power consumption and add some multiplexers to the input of some combinational logic to avoid unwanted signal transition.

Code type	n=63	n=255	n=1023	n=63	n=255	n=1023
Number of codeword	2	2	2	1	1	1
Mode	hard	hard	hard	soft	soft	soft
Power	0.0133W	0.0126W	0.0129W	0.0160W	0.0179W	0.0252W

3. Area

Area is primarily dominated by three parts : 8192 registers for LLR, adders and the associative control logic in controller.v, and 3 hard-decision decoding submodules

I had heard that some groups utilized LUTs to compute GF(2^m) inverse. This would require one LUT to convert m-tuple representation to power representation and another one LUT to convert it bach to m-tuple representation. LUTs occupy large area especially for n=1023, which require a LUT of size 1024*10. Our biggest advantage is that we do not use LUT. We use Fermat's theorem instead.

The core area is 618967.094 μm^2

Reference

Bitonic sorting: https://blog.csdn.net/xbinworld/article/details/76408595

BCH code: Shu Lin and Daniel J. Costello, "Error Control Coding: Fundamentals and Applications," Pearson, 2004.